

El Lenguaje de Programación C



Organización de Computadoras
Depto. Cs. e Ing. de la Comp.
Universidad Nacional del Sur



Copyright

● Copyright © **2011-2015** Alejandro G. Stankevicius

Copyright © **2013-2015** Dana K. Urribarri

Copyright © **2017-2021** Leonardo de - Matteis

Copyright © **2022-2025** Maximiliano Escudero

● Se asegura la libertad para copiar, distribuir y modificar este documento de acuerdo a los términos de la GNU Free Documentation License, Versión 1.2 o cualquiera posterior publicada por la Free Software Foundation, sin secciones invariantes ni textos de cubierta delantera o trasera.

● Una copia de esta licencia está siempre disponible en la página <http://www.gnu.org/copyleft/fdl.html>.



Contenidos

- Introducción al paradigma imperativo.
- Sentencias de control.
- Entrada y salida estándar.
- Pasaje de parámetros.
- Tipos de datos estructurados.
- Gestión de la memoria dinámica.
- Estructuras de datos dinámicas.
- Gestión de archivos.



Origen del lenguaje

- El **lenguaje C** fue diseñado por Dennis Ritchie en el año 1972.
 - Deriva de un lenguaje anterior llamado **B**.
- Se usó para implementar gran parte del sistema operativo **UNIX**.
- Hoy en día se lo sigue utilizando para implementar todo tipo de sistemas:
 - Por caso, el sistema operativo **GNU/Linux**.



Filosofía de diseño

- **C** es un lenguaje de programación **minimalista**.
- Intenta satisfacer múltiples objetivos:
 - Poder ser compilado usando un compilador sencillo y simple.
 - Brindar acceso de bajo nivel al hardware de la computadora.
 - Poder traducir cada instrucción de **C** en pocas instrucciones de lenguaje máquina.
 - No requerir mucha ayuda adicional en tiempo de ejecución.



Aplicaciones

El lenguaje ha sido usado para desarrollar:

- Sistemas Operativos (casi todos, GNU/Linux, OS X, Windows, Android, QNX, Solaris)
- Lenguajes de programación: Perl, PHP, Python, Ruby, etc.
- Sistemas embebidos (equipos: Arduino, Galileo, Raspberry).
- Controladores de dispositivos (drivers).
- Motores de bases de datos (MySQL, PostgreSQL, etc.).
- Sistemas de alto desempeño.



Actualidad

● Lenguajes de programación más utilizados (Ranking IEEE Spectrum).

3	C	  	94.6
C is used to write software where speed and flexibility is important, such as in embedded systems or high-performance computing.			
4	C++	  	87.0
Essentially an object-oriented version of C that proved to be a natural fit for software driven by graphical user interfaces.			

Fuente: <https://spectrum.ieee.org/top-programming-languages/>

Language Ranking: IEEE Spectrum					
Rank	Language	Type			Score
1	Python				100.0
2	Java				95.3
3	C				94.6
4	C++				87.0
5	JavaScript				79.5
6	R				78.6
7	Arduino				73.2
8	Go				73.1
9	Swift				70.5
10	Matlab				68.4



Paradigmas de programación

● A partir de las características fundamentales, los lenguajes de programación se pueden clasificar en distintos **paradigmas**:

- Paradigma imperativo.
- Paradigma funcional.
- Paradigma lógico.
- Paradigma orientado a objetos.

● El lenguaje de programación **C** pertenece al primero.



Paradigmas de programación

El paradigma al cual pertenece un lenguaje nos da indicios acerca de distintas cuestiones:

- De qué manera resulta más conveniente encarar el proceso de resolución de un problema.
- Qué abstracciones tenemos a disposición para implementar los distintos componentes de la solución propuesta.
- Finalmente, la elección del paradigma también determina la manera en que se lleva adelante la computación.



Paradigma imperativo

■ Analicemos en particular el caso del **paradigma imperativo**:

- ➔ Para resolver un problema debemos postular un **algoritmo**, esto es, una secuencia de instrucciones simples, que conduzca a la solución del mismo.
- ➔ Podemos hacer uso de variables, asignaciones y contamos con tres poderosos **constructores**: la **secuencia**, el **condicional** y la **repetición**.
- ➔ Este paradigma sanciona que la computación se lleva adelante a través de la ejecución de instrucciones relativamente simples.



Paradigma imperativo

- Este paradigma recibe su nombre por el tipo de instrucciones simples que emplea: verbos conjugados en **modo imperativo**.
- Por ejemplo:
 - printf()**: imprima con el siguiente formato los datos consignados a continuación.
 - exit()**: finalice la ejecución inmediatamente e informe el siguiente nivel de error.



Características de C

- Cuenta con **excelentes compiladores optimizantes**, por caso el **gcc**.
- El código escrito en este lenguaje es **altamente portable**.
- Ninguna característica del sistema operativo se ve reflejada en el lenguaje.



Características de C

- Todas las sentencias terminan en punto y coma.
- Los espacios en blanco y los tabuladores son ignorados.
 - Podemos (¡debemos!) indentar adecuadamente el código.
- No hay procedimientos, sólo funciones.
- No cuenta con el concepto de anidamiento de funciones.
- Solo permite el pasaje de parámetros por valor.



Características de C

- Elementos de un programa:
 - Palabras reservadas (muy pocas).
 - Variables y funciones definidas por el programador.
 - Funciones de librería estándar.
- Los bloques de código se delimitan entre llaves.
- Al igual que **UNIX**, el lenguaje **C** es sensible a mayúsculas y minúsculas (case sensitive).
 - Por ejemplo, **fact()** y **Fact()** denotan a dos funciones diferentes.



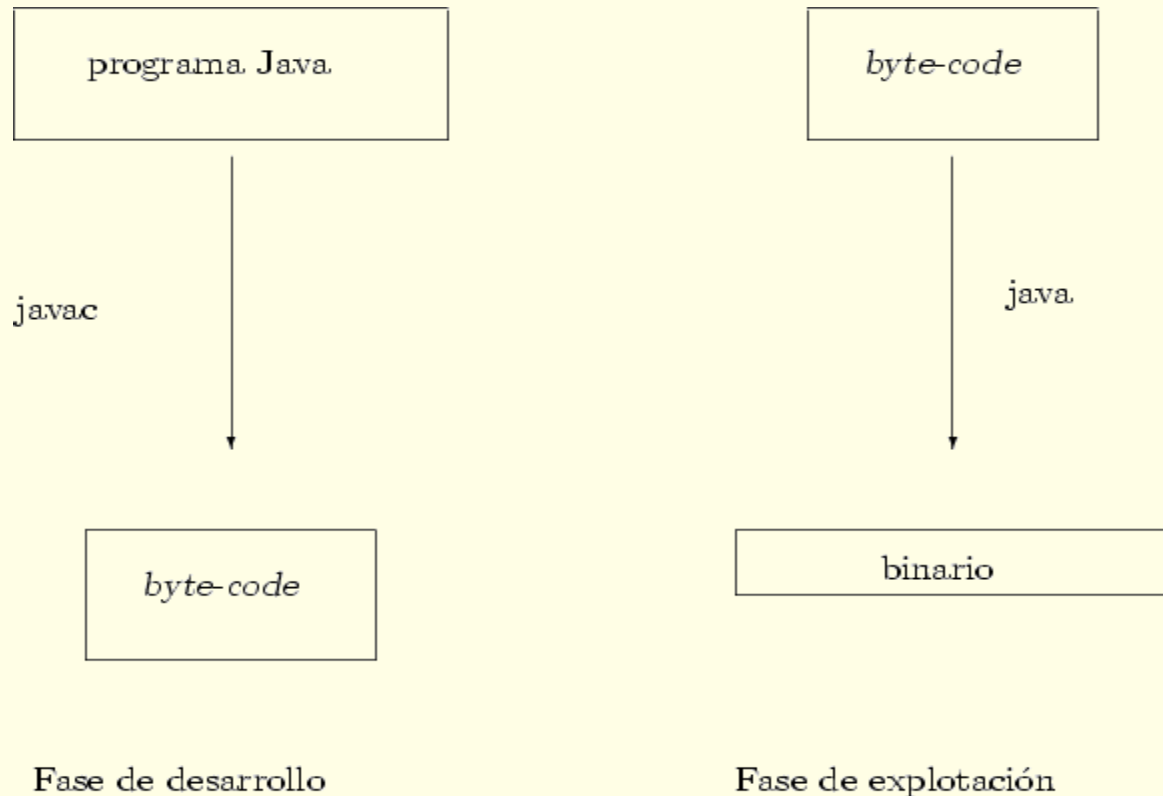
Características de C

- El lenguaje **C** es un lenguaje **compilado**.
 - ➔ Ejemplos: Pascal o Modula2.
- Otros lenguajes de programación son en cambio **interpretados**.
 - ➔ Ejemplos: Prolog o Haskell.
- Por último, también existen lenguajes **compilados e interpretados a la vez**:
 - ➔ Ejemplo: Java.



Características de C

● Ejemplo: Java.



Características de C

●Java:

●Compilado: código fuente es traducido a un lenguaje objeto llamado código máquina (bytecodes).

●Interpretado: el código máquina puede ser ejecutado en casi cualquier plataforma la cual debe tener un interprete para su ejecución.

●Plataformas compatibles: <https://docs.oracle.com/cd/E19146-01/820-5658/gduwv/index.html>



Compiladores vs. intérpretes

● Lenguajes interpretados:

- Las instrucciones se transforman en lenguaje máquina a medida que se ejecuta el programa.
- Los pequeños cambios pueden probarse rápidamente.

● Lenguajes compilados:

- Los programas son traducidos en su totalidad a lenguaje máquina antes de ser ejecutados.
- Para probar un pequeño cambio se debe compilar todo nuevamente.



Nuestro primer programa

El clásico “hola mundo”, en esta oportunidad implementado en **C**:

```
#include <stdio.h>
```

```
int main() {
```

```
    printf("Hola mundo!!\n");
```

```
    return 0;
```

```
}
```



Comentarios

- Los **comentarios** pueden ocupar una o varias líneas:

```
int main() { // Comentario de una línea.  
    /* Esto es un comentario  
    de varias líneas. */  
    return 0;  
    // Otro comentario de una línea.  
}
```



Finalización de un programa

Un programa finaliza:

- ➔ Al llegar al final de la función `main()`.
- ➔ Cuando la función `main()` invoca un `return`.
- ➔ Al ejecutar la función `exit()` (la cual reside en la librería estándar `stdlib`).

```
#include <stdlib.h>
```

```
int main() {
```

```
    exit(0);
```

```
    return 1; // Esto no se ejecutará
```

```
}
```



Tipos elementales

● Los tipos elementales son:

- Enteros (**int**).
- Reales en precisión simple (**float**).
- Reales en precisión doble (**double**).
- Caracteres y enteros de 1 byte (**char**).
- Punteros (*****).

● Nótese que no existen los **booleanos** ni las **cadenas de caracteres** como tipos elementales.



Modificadores

Los tipos elementales admiten distintos modificadores:

- ➔ **unsigned**: para representar sólo valores positivos.

- ➔ Ej: **unsigned int contador;**

- ➔ **signed**: para representar valores positivos y negativos (este es el modificador activo por defecto).

- ➔ Ej: **signed char letra;**

- ➔ **long**: para representar enteros largos (sólo puede aplicarse al tipo **int**).

- ➔ Ej: **long int balance;**



Declaración de variables

- Declaración simple:

```
char letra;
```

```
unsigned int contador;
```

- Declaración múltiple:

```
char letra, inicial;
```

```
unsigned int i, j, k;
```

- Declaración e inicialización:

```
char letra = 'A';
```

```
unsigned int i = 1, j = 10;
```



Alcance de una declaración

- Recordemos que en el lenguaje **C** no es posible anidar funciones.
- En consecuencia, sólo hay dos alcances posibles para una declaración:
 - Alcance global: La declaración es visible desde todas las funciones del programa.
 - Alcance local: La declaración sólo es visible dentro de la función en la que aparece y tiene precedencia por sobre las declaraciones globales.



Alcance de una declaración

```
int x, y;  
  
int main() {  
    float x, z;  
  
    /* Aquí y sigue siendo entero, pero x y  
    z ahora son reales */  
  
}  
  
/* x e y vuelven a ser enteros, y además z  
no existe por fuera de la función main()*/
```



Expresiones constantes

Formulación de expresiones constantes:

- Una constante real (**float**) se puede expresar tanto en notación decimal (**2.56**) como en notación científica (**2.45E-4**).
- Una constante de tipo doble precisión (**long**) se denota agregando una **L** al final (**200L**).
- Una constante de tipo carácter (**char**) se define directamente usando comillas simples (**'a'**).
- Una constante de tipo cadena de caracteres se define usando comillas dobles (**"hola!"**).



Expresiones constantes

Las constantes de tipo carácter especiales se definen usando **secuencias de escape**:

- `'\n'`: línea nueva.
- `'\r'`: retorno de carro.
- `'\t'`: tabulador.
- `'\\'`: barra invertida.
- `'\0'`: carácter nulo.
- `'\nnn'`: carácter **ASCII** nnn (en octal).
- `'\xnn'`: carácter **ASCII** nn (en hexadecimal).



Conversión explícita de tipos

La **conversión explícita de tipos** (type casting) de expresiones y variables se denota de la siguiente forma:

```
int a; float b; char c;
```

```
b = 65.0; /* probar qué pasa si acá  
ponemos directamente 65 */
```

```
a = (int) b; // a ahora vale 65
```

```
c = (char) a; /* c ahora vale 'A'  
(esto es, el ASCII 65) */
```



Operadores aritméticos

- **C** usa como **operador de asignación** al signo igual (=).
- Los **operadores aritméticos básicos** disponibles son los usuales:
 - Suma (+)
 - Resta (-)
 - Multiplicación (*)
 - División (/)
 - Módulo o resto (%)



División entera vs. real

El tipo del resultado de la operación de división depende enteramente del tipo de los operandos involucrados.

Por ejemplo:

4 / 3 da 1 (entero)

4.0 / 3 da 1.333 (real)

4 / 3.0 da 1.333 (real)

4.0 / 3.0 da 1.333 (real)



Pre y post incremento

• Los operadores unarios (**++**) y (**--**) representan las operaciones de **incremento** y **decremento** respectivamente.

• **x++**; equivale a la asignación **x = x + 1**;

• **y--**; equivale a la asignación **y = y - 1**;

• Por caso:

a=3; b=a++; // a vale 4, b vale 3

a=3; b=++a; // a vale 4, b vale 4

• No cualquier cosa es válida: **--b=a++**;



Variantes de asignación

Existen diversas variantes del **operador de asignación**:

➔ $x += y;$ es equivalente a $x = x + y;$

➔ $x -= y;$ es equivalente a $x = x - y;$

➔ $x *= y;$ es equivalente a $x = x * y;$

➔ $x /= y;$ es equivalente a $x = x / y;$

El lado izquierdo de una asignación debe necesariamente ser del tipo adecuado.

➔ Caso contrario, usar una conversión explícita de tipo.



Operaciones a nivel de bit

El lenguaje **C** brinda un conjunto de **operadores numéricos a nivel de bit**:

- Negación: $\sim x$;
- Conjunción: $x \& y$;
- Disyunción: $x | y$;
- Disyunción exclusiva: $x \wedge y$;
- Desplazamiento a izquierda: $x \ll n$;
- Desplazamiento a derecha: $x \gg n$;



Operaciones a nivel de bit

`int x = 26;`

`00011010 = 26`

`int y = 61;`

`00111101 = 61`

`x & y;`

`00011000 = 24`

`x | y;`

`00111111 = 63`

`~x;`

`11100101 = -27`

`x >> 2;`

`00000110 = 6`



Operadores relacionales

● Los **operadores relacionales** disponibles son:

- ➔ Igual (**==**) .
- ➔ Distinto (**!=**).
- ➔ Mayor (**>**) y mayor o igual (**>=**).
- ➔ Menor (**<**) y menor o igual (**<=**).

● El resultado de una comparación **es un entero**, el cual se interpreta de la siguiente manera:

- ➔ El cero denotará el valor falso.
- ➔ Cualquier otro valor denotará el valor verdadero.



Operadores lógicos

• Sobre los datos de tipo **booleano** (es decir, los enteros), se definen los siguientes operadores:

- La conjunción lógica (**&&**).
- La disyunción lógica (**||**).
- La negación lógica (**!**).

• Por caso:

```
/* ¿a cuánto evalúa la siguiente  
expresión?*/
```

```
rsta = (3 > 2 || 5 == 4) && !1;
```



Expresiones en corto-circuito

- El lenguaje **C** implementa un modelo de **evaluación perezosa** para las expresiones booleanas.

- Por caso:

```
a = (3 > 2 || w == 4) ;  
/* la subexpresion w == 4  
nunca llega a evaluarse */
```

```
// ¿por qué razón?
```



Contenidos

- Introducción al paradigma imperativo.
- Sentencias de control.
- Entrada y salida estándar.
- Pasaje de parámetros.
- Tipos de datos estructurados.
- Gestión de la memoria dinámica.
- Estructuras de datos dinámicas.
- Gestión de archivos.



Sentencia condicional

La **sentencia condicional** tiene la siguiente sintaxis:

```
if (condición) {  
    // código a ejecutarse si  
    // la condición se satisface  
} else {  
    // código a ejecutarse en  
    // caso contrario.  
}
```



Sentencia condicional

```
#include <stdio.h>

main() {
    int a = 2;
    if (a % 2) {
        printf("dos es impar\n");
    } else {
        printf("dos es par\n");
    }
}
```



Expresiones condicionales

- Las construcciones del siguiente tipo

```
if (condición)
    rsta = exp-1;
else
    rsta = exp-2;
```

- se pueden abreviar en el lenguaje **C** empleando expresiones condicionales.

- Sintaxis:

```
rsta = condición ? exp-1 : exp-2;
```



Expresiones condicionales

```
#include <stdio.h>

main() {
    int a = 2;
    printf("dos es %s\n",
        a % 2 ? "impar" : "par");
}
```



Sentencia de repetición

La sentencia de repetición **for** es altamente flexible en este lenguaje:

```
for (cont = 0; cont < tope; cont++) {  
    /* bloque de sentencias que se  
    repiten mientras se cumpla la  
    condición de permanencia */  
}
```



Sentencia de repetición

```
#include <stdio.h>

main() {
    int cont;
    for (cont = 0; cont < 10; cont++) {
        printf("%i al cuadrado es %i\n",
            cont, cont * cont);
    } // ¿llegará hasta 10?
}
```



Sentencia de repetición

■ También es posible repetir un fragmento de código mientras se cumpla una cierta condición:

```
while (condición) {  
    /* bloque de sentencias que se repiten  
    cero o más veces mientras se cumpla la  
    condición de permanencia */  
}
```



Sentencia de repetición

```
#include <stdio.h>

main() {
    int cont = 0;
    while (cont++ < 10) {
        printf("%i al cuadrado es %i\n",
            cont, cont * cont);
    } // ¿ahora llegará hasta 10?
}
```



Sentencia de repetición

● Por último, existe una segunda variante para la sentencia de repetición basada en la satisfacción de una condición:

```
do {  
    /* bloque de sentencias que se repite  
    al menos una vez mientras se cumpla la  
    condición de permanencia */  
} while (condición);
```



Sentencia de repetición

```
#include <stdio.h>

main() {
    int cont = 0;
    do {
        printf("%i al cuadrado es %i\n",
            cont, cont * cont);
        cont++;
    } while (cont < 10); // ¿y ahora?
```



Sentencias break y continue

■ Al estar dentro del ámbito de una sentencia de repetición, se puede hacer uso de dos sentencias de control especiales:

- **break**: esta sentencia permite romper una repetición, retomando la ejecución en la instrucción inmediata siguiente.
- **continue**: esta sentencia permite indicar que la iteración actual ya fue completada y que se desea considerar la siguiente iteración.



Sentencias break y continue

```
#include <stdio.h>

main() {
    int cont = 0;
    do {
        printf("%i visitado\n", cont++);
        if (cont > 10) break;
    } while (1); // ¿cuándo corta?
}
```



Sentencias break y continue

```
#include <stdio.h>
```

```
main() {
```

```
    int cont = 0;
```

```
    for (int cont = 0; cont < 10; cont++)
```

```
{
```

```
    if (cont == 3) continue;
```

```
    printf("%i visitado\n", cont);
```

```
}
```



Alternativas múltiples

Las alternativas múltiples en la ejecución se modelan por medio de la siguiente sentencia:

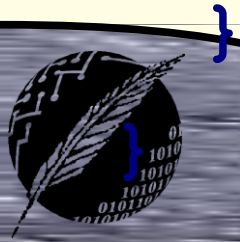
```
switch (alternativa) {  
    case caso-1: { // bloque-1 }  
    case caso-2: { // bloque-2 }  
    ...  
    case caso-n: { // bloque-n }  
    default: { // bloque por defecto }  
}
```



Alternativas múltiples

```
#include <stdio.h>

main() {
    char letra = '\x42';
    switch (letra) {
        case 'A': { printf("A\n"); }
        case 'B': { printf("A o B\n");
                    break; }
        case 'C': { printf("C\n"); }
    }
```



Contenidos

- Introducción al paradigma imperativo.
- Sentencias de control.
- Entrada y salida estándar.
- Pasaje de parámetros.
- Tipos de datos estructurados.
- Gestión de la memoria dinámica.
- Estructuras de datos dinámicas.
- Gestión de archivos.



Entrada y salida estándar

Las funciones de **entrada/salida estándar** en **C** no pertenecen al lenguaje en sí, sino que son provistas por diversas funciones de librería.

- Recordemos que uno de los objetivos de diseño es lograr una alta portabilidad, por lo que el lenguaje debe ser absolutamente independiente del sistema operativo.

Por caso,

- Funciones de entrada: **getchar()**, **scanf()**, etc.
- Funciones de salida: **putchar()**, **printf()**, etc.



La función printf()

● La invocación a la función **printf()** guarda la siguiente sintaxis:

```
printf(  
    formato-a-utilizar,  
    exp-1, exp-2, ..., exp-n);
```

● donde:

- El **formato** a utilizar es una cadena que describe cómo mostrar la información.
- Las **expresiones** son los datos que se desean mostrar.



La función printf()

```
int x = 3; float y = 23.0; char z = 'A' ;
```

```
printf("Hola mundo!!\n") ;
```

```
printf("x vale %d\n", x) ;
```

```
printf("y vale %f,\n...y z vale %c.\n",  
      y, z) ;
```



Expresiones de formato

En la **E/S** estándar, se puede hacer uso de las siguientes expresiones de formato (entre otras):

- ➔ **%c**: carácter.
- ➔ **%i**: entero.
- ➔ **%d**: entero decimal.
- ➔ **%x**: entero hexadecimal.
- ➔ **%f**: real.
- ➔ **%p**: puntero.
- ➔ **%s**: cadena de caracteres.



Expresiones de formato

● Existen muchas otras expresiones que controlan otros aspectos del formato:

- ➔ La precisión (número de decimales).
- ➔ La justificación (a izquierda o a derecha).
- ➔ Poder escribir ciertos caracteres especiales (por ejemplo, el carácter %).

● Consultar la referencia de esta función en la bibliografía recomendada:

- ➔ “The C Programming Language”, escrito por B. Kernighan y D. Ritchie.



La función scanf()

● La invocación a la función **scanf ()** guarda la siguiente sintaxis:

```
scanf(  
    formato-a-utilizar,  
    dest-1, dest-2, ..., dest-n);
```

● donde:

- El **formato** a utilizar es una cadena que describe cómo recibir la información.
- Las **expresiones** son los destinos de lo ingresado.



La función scanf()

```
int x; float y; char z;
```

```
printf("Ingrese un número entero: ");
```

```
scanf("%d", &x); // ¡x se modifica!
```

```
printf("\nOtro entero y un real: ");
```

```
scanf("%u %f", &x, &y);
```

```
printf("\nAhora un caracter: ");
```

```
scanf("%c", &z);
```



Contenidos

- Introducción al paradigma imperativo.
- Sentencias de control.
- Entrada y salida estándar.
- Pasaje de parámetros.
- Tipos de datos estructurados.
- Gestión de la memoria dinámica.
- Estructuras de datos dinámicas.
- Gestión de archivos.



Pasaje de parámetros

■ Si bien **C** no cuenta con pasaje de parámetros por referencia, cuenta con un operador que **permite simular** ese pasaje de parámetros.

- El operador **&** determina la dirección en memoria de la expresión a su derecha.

■ Por caso:

```
int a = 5;
```

```
printf("a vale %i\n", a);    // ¿por qué %i se
```

```
printf("&a vale %p\n", &a);  // cambia por %p?
```



Pasaje de parámetros

● Pasaje por valor o por copia:

- Al pasar un parámetro por valor estamos pasando una copia del contenido, por lo que las modificaciones que haga la rutina en esa copia no se verán reflejadas en el original.

● Pasaje por referencia:

- Al pasar un parámetro por referencia estamos mostrando como acceder a ese argumento pues estamos pasando la dirección en memoria del mismo. Naturalmente, las modificaciones que haga la rutina se ven inmediatamente reflejadas en el original.



Definición de funciones

● La **definición de una función** guarda la siguiente estructura:

```
tipo nombre (parámetros) {
```

```
// cuerpo de la función.
```

```
}
```



Definición de funciones

- Una función se invoca al suministrar **valores actuales** a sus **parámetros formales**.
 - Recordemos, los parámetros formales son los que figuran en la definición de una función y los actuales son los usados en una invocación en concreto.
- Los parámetros siempre se pasan por valor.
 - Hemos visto que el programador puede simular el pasaje por referencia cuando así lo requiera.
- El resultado de la invocación a una función se especifica por medio de la sentencia **return**.



Definición de funciones

- Los **procedimientos**, si bien no contemplados en el lenguaje, se pueden definir indicando que una cierta función no retorna resultado.
 - Para esto se debe hacer uso del tipo especial **void**.
- El control del número y tipo de los parámetros es mínimo.
- Las funciones admiten **llamadas recursivas**.



Definición de funciones

```
int fact(int n) {  
    int rsta = 1;  
    while (n > 1)  
        rsta *= n--;  
    return rsta;  
}  
  
int main() {  
    printf("%d! = %d\n", 5, fact(5));  
}
```



Definición de funciones

```
void f(int* a, int b) {  
    *a = b;  
}
```

```
int main() {  
    int x = 5;  
    int y = 1;  
    printf("x = %d\n", x);  
    f(&x, y);  
    printf("x = %d", x);  
    return 0;  
}
```



Definición vs. declaración

● **C** sólo permite invocar a funciones previamente definidas.

→ ¿cómo hacer para definir programas que hagan uso de una **recursión cruzada**?

● Es posible separar espacialmente la declaración de una función de su definición:

→ La **definición** explicita el código asociado a la función.

→ La **declaración** sólo explicita la cabecera o prototipo de la misma.



Definición vs. declaración

```
int impar(int n); // prototipo de impar()
```

```
int par(int n) { // definición de par()  
    return !n ? 1 : impar(--n);  
}
```

```
int impar(int n) { // definición de impar()  
    return !n ? 0 : par(--n);  
}
```



¿Preguntas?

